

Automação da Atualização de ERP

Apresentação Executiva, Técnica e de Status Operacional

Objetivo Estratégico:

Eliminar o processo manual e repetitivo de atualização de servidores via AnyDesk, reduzindo custos operacionais, mitigando riscos humanos de corrupção de banco e permitindo escalar a carteira de clientes com monitoramento em tempo real.

Gestor de Atualizações + Agente Atualizador ERP

Bredas Sistemas · Setembro de 2026

Preparado por: Antonio Salomão

Resumo Executivo:

Como transformar um processo de atualização de ERP hoje 100% manual, demorado e arriscado (via AnyDesk máquina por máquina) em uma esteira segura, automática, padronizada e monitorada em tempo real por um painel central.

1. O Problema que Motivou o Projeto

Hoje, cada vez que a Bredas lança uma nova versão do ERP (seja com melhorias fiscais, novas telas ou correções), atualizá-la nos clientes é um processo totalmente manual e repetitivo.

Como funciona hoje (o gargalo operacional)

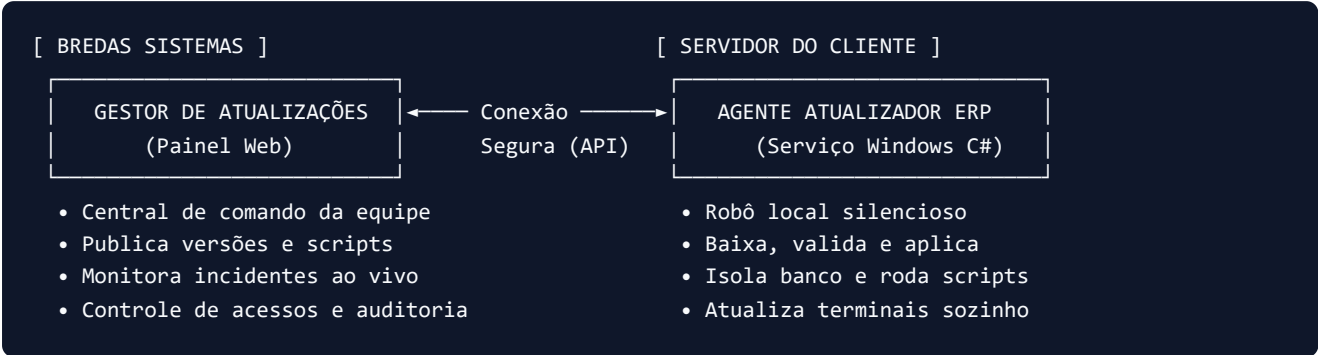
1. Alguém da equipe de suporte precisa agendar um horário com o cliente.
2. Abre o **AnyDesk** ou TeamViewer e conecta no servidor do cliente.
3. Executa manualmente uma sequência de 8 passos delicados:
 - Pedir para todos os usuários saírem do ERP;
 - Fazer backup de segurança do banco Firebird;
 - Renomear executáveis antigos na pasta do sistema;
 - Copiar os novos executáveis baixados manualmente;
 - Abrir ferramenta de banco e rodar scripts SQL de atualização;
 - Conferir se não deu nenhum erro de tabela ou coluna;
 - Fazer um novo backup pós-atualização;
 - Abrir o sistema em uma estação para testar o login.
4. Repete esse mesmo procedimento em **cada um dos clientes da carteira**.

As dores e custos desse modelo

- **Tempo desperdiçado da equipe:** Cada cliente consome de 30 a 60 minutos de um técnico sênior. Em uma versão distribuída para 50 clientes, são mais de 30 a 40 horas de suporte apenas copiando arquivos e olhando barras de progresso.
 - **Risco humano no banco de dados:** Um passo esquecido, uma queda de conexão no AnyDesk durante a execução de um script SQL ou rodar um script na ordem incorreta pode corromper o banco de dados do cliente e parar a empresa dele.
 - **Falta de controle central:** Não havia um lugar único e confiável para responder perguntas básicas como: "*Quantos clientes já estão na versão 2026.08?*", "*Quem ainda está desatualizado?*", "*Quando foi a última vez que o cliente X recebeu atualização?*". Essas informações ficavam perdidas em planilhas ou na memória de cada técnico.
 - **Barreira de crescimento:** A carteira de clientes não consegue dobrar ou triplicar sem ter que dobrar a equipe de suporte apenas para atualizar versões.
-

2. Visão Geral da Solução: Os Dois Pilares

Para resolver a raiz do problema, o projeto foi dividido em dois componentes especializados que trabalham integrados através de uma conexão segura:



Componente	O que é	Onde roda	Papel no negócio
Gestor de Atualizações (A Central de Comando)	Painel web corporativo acessível pelo navegador.	Servidor interno da Bredas.	Onde a equipe cadastra versões novas, gerencia clientes, controla acessos e acompanha em tempo real o status de cada servidor instalado.
Agente Atualizador ERP (O Assistente Local)	Serviço Windows leve e silencioso construído em C# (.NET 8).	No servidor local de cada cliente (ao lado do banco Firebird).	Executa sozinho todo o procedimento manual: baixa a versão, valida integridade, aplica scripts SQL, atualiza os executáveis e reporta o resultado de volta para o painel.

Como os dois conversam

O agente local consulta periodicamente a API do Gestor Web através da internet:

"Existe versão nova para o sistema deste cliente?"

- Se não houver nada novo, ele volta a dormir sem consumir recursos.
- Se houver uma versão nova publicada, ele baixa o pacote criptografado, valida se o arquivo não veio corrompido, executa a atualização com segurança total e reporta: *"Atualização concluída com sucesso em 4 minutos"*.
- Se houver qualquer falha ou se o agente ficar sem comunicação por mais de 24 horas, o painel web acende um **alerta vermelho imediato** e avisa a equipe no canal do **Discord**.

3. O Fluxo das 4 Fases: Segurança em Primeiro Lugar

O maior receio de qualquer empresa de software ao automatizar atualizações é:

"E se o sistema atualizar no meio de uma venda no caixa?" ou "E se der erro no banco de dados e o cliente parar?"

Por isso, o agente foi projetado com uma esteira estrita de **4 Fases sequenciais blindadas**:

└ Fase 1: Preparo Silencioso (Download e Validação)

- O agente baixa o pacote da versão nova em segundo plano pela internet.
- **Validação Criptográfica (SHA-256)**: O agente confere se o arquivo baixado é exatamente idêntico ao gerado na Bredas. Se a internet oscilar ou o arquivo corromper, o pacote é descartado imediatamente.
- O cliente continua trabalhando normalmente sem perceber nenhuma lentidão.

└ Fase 2: Permissão e Respeito ao Cliente (Autorização)

- O sistema **nunca derruba o cliente de surpresa**.
- Quando o pacote está pronto, o próprio ERP Delphi avisa o usuário do cliente em momento oportuno: *"Uma nova versão está pronta. Deseja aplicar a atualização agora?"*
- O cliente escolhe a melhor hora (por exemplo, no final do expediente ou no intervalo do almoço). Quando ele confirma, o ERP grava a autorização e o agente assume o comando.

└ Fase 3: Execução Crítica com Escudo Total (Banco Firebird & Scripts)

- O agente desconecta os usuários externos do banco via comando nativo (`gfix`).
- **Backup de Segurança Obrigatório**: Antes de encostar em qualquer dado, o agente gera um backup completo do banco de dados local.
- **Aplicação Controlada dos Scripts SQL**: O motor próprio do agente (`ScriptRunnerService`) aplica os scripts de banco um por um, registrando cada arquivo executado.
- **Rede de Segurança Automática (Rollback Garantido)**: Se qualquer comando SQL falhar ou a energia cair, o agente desfaz as mudanças, restaura o backup inicial e religa o banco de dados. **O cliente nunca amanhece com o sistema quebrado.**

└ Fase 4: Distribuição Automática para os Terminais

- O agente injeta os novos executáveis compilados na tabela central (`BEXE.fdb`).
 - Os computadores e caixas da loja/escritório baixam os executáveis atualizados diretamente da rede local ao abrirem o ERP. Não é necessário atualizar máquina por máquina.
 - O agente envia o comprovante de sucesso para o Gestor Web da Bredas.
-

4. O que Já Está Pronto e em Funcionamento Hoje

O projeto não está no papel: ele já possui funcionalidades maduras, seguras e testadas em produção.

1. Painel Gestor de Atualizações (Web) — Em uso diário na Bredas

- **Plataforma Moderna Multiusuário:** Substituiu o antigo aplicativo desktop local por uma aplicação web robusta, rápida e acessível por toda a equipe.
- **Segurança e Perfis de Acesso (RBAC):**
 - **Administrador** : controle total, aprovação de versões, gestão de usuários, backups e chaves de API.
 - **Operador** : rotinas de atendimento, cadastro de clientes, agendamentos e envio de rascunhos.
 - **Consulta** : visualização de relatórios e status sem permissão de alteração.
- **Publicação Blindada de Versões:** Upload em streaming com cálculo de integridade SHA-256 em tempo real. O banco só publica a versão se o arquivo físico estiver 100% íntegro no disco.
- **Painel de Distribuição ao Vivo:** Acompanhamento de toda a base de agentes instalados com fila de prioridade visual: erros em vermelho, agentes sem contato em amarelo e clientes em dia em verde.
- **Centro de Saúde Operacional:** Tela de diagnóstico em tempo real que afere a integridade física do banco de dados (`PRAGMA integrity_check`), uso de memória e cópias de segurança.
- **Matriz Comparativa na Consulta:** Ao buscar um cliente, a tela exibe instantaneamente:
`Sistema | Versão Instalada | Versão Publicada | Estado | Último Contato`
- **Pesquisa Global (`Ctrl+K`):** Localização instantânea de clientes, versões no ar, incidentes de agentes e atalhos rápidos de operação.
- **Alertas no Discord:** Notificação em tempo real no canal da equipe sempre que uma versão é publicada ou um cliente reporta falha.

2. Agente Atualizador ERP (C#) — Pré-Piloto Homologado

- **Código 100% Compilado e Estruturado:** Construído em .NET 8 como serviço nativo do Windows, iniciando sozinho junto com o computador.
 - **Testado com Banco de Produção Real:** O ciclo completo (Fase 1 → Fase 3 → Fase 4) foi testado e aprovado contra uma cópia real do banco Firebird com **366 tabelas e mais de 1.000 scripts SQL reais** do `B_Vendas` .
 - **Formato dos Executáveis Confirmado:** A injeção de arquivos no `BEXE.fdb` foi validada campo a campo contra um banco real de cliente.
 - **Rollback Automático Comprovado:** Cenários de erro forçado foram testados em laboratório, e o agente restaurou o backup e reverteu o banco com sucesso em todas as tentativas.
-

5. Riscos Mapeados e Como Cada Um Foi Blindado

A auditoria técnica levantou antecipadamente os riscos operacionais e implementou travas de engenharia para cada um:

Risco Mapeado	O que poderia acontecer	Como o sistema foi blindado	Situação Atual
Queda de energia ou reinício no meio da atualização	Banco Firebird ficar bloqueado ou inacessível.	O agente detecta no boot que uma atualização foi interrompida e retoma o processo com segurança a partir do backup inicial.	Corrigido e Testado
Falha em script SQL do banco	Cliente ficar com schema incompleto ou dados corrompidos.	O agente interrompe o processo na hora, restaura o backup prévio e devolve o banco ao ar no estado anterior funcional.	Corrigido e Testado
Arquivo corrompido no download	Executável quebrado ser gravado na pasta do cliente.	Validação rígida de hash SHA-256 antes da extração. Se 1 byte for diferente, o arquivo é rejeitado.	Corrigido e Testado
Operador publicar versão errada	Um pacote não testado ser enviado a todos os clientes.	Permissão restrita a Administradores com transação atômica e verificação física dos pacotes em disco.	Corrigido e Testado
Scripts antigos serem reaplicados	Erro de "tabela ou coluna já existente" no banco.	O agente consulta o histórico da tabela <code>SCRIPTS</code> e pula qualquer arquivo que já tenha sido aplicado no passado.	Mitigado (Requer triagem no piloto)
Atualização ocorrer durante venda no cliente	Caixa travar na frente do consumidor.	O agente só executa se houver a confirmação explícita do usuário no ERP (Fase 2).	Pendente de tela no Delphi

6. Ganhos do Negócio (Retorno sobre o Investimento)

Investir na finalização e ativação deste projeto traz retornos claros para a operação da Bredas:

1. Eficiência da Equipe de Suporte

- Elimina a necessidade de técnicos ficarem presos no AnyDesk executando tarefas manuais repetitivas.
- A equipe passa a atuar de forma **proativa e estratégica**, focando em dúvidas de clientes, novos módulos e atendimento de qualidade.

2. Velocidade de Entrega de Valor

- Hoje, distribuir uma versão crítica (como uma mudança urgente de legislação tributária da SEFAZ) para toda a base pode levar dias ou semanas.
- Com a automação, a versão é publicada uma única vez na central, e os servidores dos clientes se atualizam sozinhos de forma coordenada.

3. Segurança e Fim das "Surpresas"

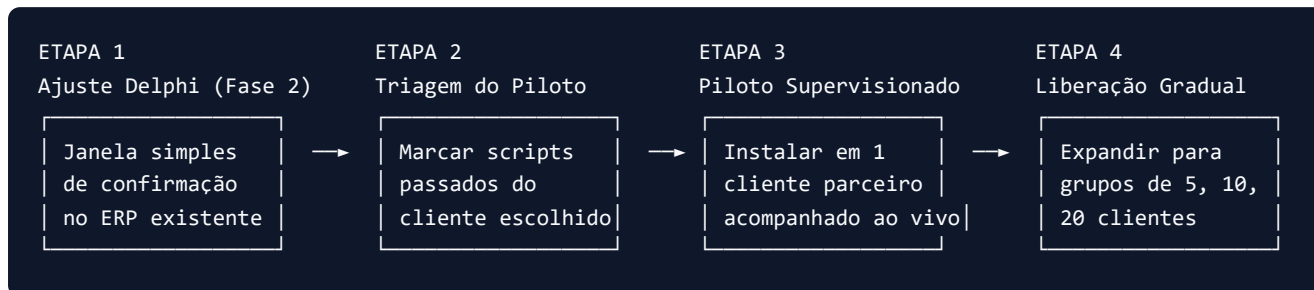
- Padronização absoluta: todos os clientes recebem o mesmo procedimento verificado, sem variações individuais causadas por cansaço ou pressa de operadores.
- Se um cliente tiver qualquer oscilação ou falha, o painel da Bredas avisa a equipe antes mesmo de o cliente perceber e abrir chamado.

4. Escalabilidade Real

- A empresa pode crescer sua carteira de clientes de forma acelerada sem que o custo de suporte operacional cresça na mesma proporção.
-

7. O que Falta para o Piloto e Próximos Passos

O projeto está na reta final para entrar em campo. Para ligar o primeiro cliente piloto com segurança absoluta, restam apenas duas etapas bem delineadas:



1. Etapa 1: Janela de Confirmação no ERP Delphi (Fase 2)

- **O que é:** No código Delphi do ERP, adicionar uma checagem simples na inicialização: se houver registro com status `PENDENTE` na tabela `SYS_ATUALIZACAO`, exibe uma mensagem para o cliente: *"Existe uma atualização disponível. Deseja aplicar agora? [Sim] [Lembrar mais tarde]"*.
- Ao clicar em Sim, o Delphi grava `AUTORIZADO` e fecha o sistema para o agente rodar.

2. Etapa 2: Triagem de Scripts do Cliente Escolhido

- Selecionar um cliente parceiro de baixo risco para ser o **Cliente Piloto 01**.
- Fazer uma rápida conferência nos scripts que esse cliente já rodou no passado, garantindo que a tabela `SCRIPTS` dele esteja alinhada.

3. Etapa 3: Execução Supervisionada do Piloto

- Instalar o serviço do agente no servidor desse cliente piloto.
- Agendar uma atualização e acompanhar o ciclo completo em tempo real pelo Painel Web e pelos logs do Discord.

4. Etapa 4: Expansão Controlada

- Com o piloto validado, iniciar a instalação gradual nos demais clientes da carteira em lotes controlados.

8. Repositórios e Governança

O código-fonte do projeto está organizado em dois repositórios seguros com versionamento git completo:

- **Gestor de Atualizações (Painel Web + API Central):**

`github.com/antoniossalomao/gestor_atualizacao`

- **Agente Atualizador ERP (Serviço Windows C#):**

`github.com/antoniossalomao/atualizador_automatico`